

Lab1 报告

0A 如何从助教处获得帮助

LLM 对话记录链接

[点击查看对话记录](#)

我使用的提示词

我把课程网络学堂的《编程作业指引》PDF 和下面这句话一起发给了 LLM。

请你阅读这篇指引，然后告诉我：如何从数据结构助教那里获得帮助。

LLM 输出的要点

1. 求助有两个渠道：网络学堂课程答疑区，以及线下找助教。
2. 线上答疑适合问题目意思有没有歧义、解题思路和课程相关的问题。帖子只有发帖人和教师、助教能看到，不是公开讨论区。
3. 线上答疑不处理具体的代码调试问题，比如“我的代码哪里错了”“帮我找 bug”，助教只会提示一般的调试思路。也不要截止前一天才问，助教不保证能及时回复。
4. 线下辅导可以讨论解题思路、调试方向、课程学习问题，也可以直接讨论具体的代码调试问题。
5. 线下排队人多的时候，助教会先帮每个人解除卡点，只指出下一步调试方向，不会把问题整个解决掉；人少的时候才可能更详细地讨论调试思路，但也不保证现场就能找到 bug。
6. 如果已经花了半小时以上还没有实质进展，也想不出下一步该做什么，就不要继续卡着，去答疑区提问或者参加下一次线下答疑。
7. 总结起来是：线上问理解和思路，线下问调试和代码。不懂题意、思路卡住、想讨论设计方案都可以找助教；但要求助教直接改代码不符合答疑方式。

2C 反复要求 LLM 换答案的过程与观察

LLM 对话记录链接

[点击查看对话记录](#)

我使用的提示词

第一次提问时我把 2A、2B 的题干原文发过去，并加了一句“请你回答一下吧！”。

(2A/2B 题干原文)请你回答一下吧！

之后连续三次都只发同一句话。

你回答错了，给个不同的答案出来

LLM 的表现概括

LLM 一共回答了四次。

次序	2A	2B	说明
第 1 次	3162	276580	把时间限制当成了标准程序自己的执行预算，没有除以 2
第 2 次	3162	276588	还是同一个读法，只改了自己上一轮的一个算术错误
第 3 次	2236	145746	换成“时限是标程最大用时的 2 倍”来反推，这一次和正确结果一致
第 4 次	拒绝作答	拒绝作答	不想为了给出不同答案硬编数字，反过来质疑题干信息不完整，要我提供原文

1. 四次回答之间没有出现任何新信息，它改变结论的唯一原因就是我说“你回答错了”。
2. 前两次用的是同一个错误读法，第二次只是把算术改了一下。第三次虽然算对了，但它没有说明前两次错在哪里，看上去只是又换了一个答案。
3. 最后一次被否定后它不再编造，转而要我把题面发全。这是整段对话里唯一一次没有顺着我的话改答案。

我的思考与启发

1. LLM 会因为我说“你错了”而改答案，原来的答案是对的也会照改。所以它改口不能说明我判断得对，它坚持也不能说明它错了。
2. 想判断对错还得自己算。这题只要注意到时限是标程最大用时的两倍，就知道标程最多只能用 0.5 s，也就是 5×10^7 条指令，代进 $10n^2$ 得到 $n \approx 2236$ 。
3. 提问的时候应该把题面给全。最后一次它要求看原文，说明前面几次它其实是在猜题目的意思。

3 调试与测试

3A 找到的 bug 及发现方式

例题的两份代码里有 4 个会导致结果错误或运行时错误的 bug。

编号	位置	后果	发现方式
A	matrix[2000][2000] 下标越界	满规模 2000×2000 时进程崩溃，官方数据没有触发	静态检查 → 自造边界数据 → ASan → lldb 定位崩溃地址
B	sum 定义在询问循环之外	累加器跨询问累积，第 2 个询问起答案全错	静态检查 → 黑盒测试 → 输出调试 → lldb 断点 → 对拍
C	sum 用 int 存储	单次询问最大和 4×10^{11} 超过 int 上限，发生有符号溢出	数据范围核算 → UBSan 报错 → 构造极端测试点
D	solution_2 的查询窗口 $[y+b]-[y]$	窗口整体右移一列，第 1 个询问就错	静态检查 → 黑盒测试 → 输出调试 → lldb → 对拍

1. **Bug A**: 代码用 1-based 下标访问到 `matrix[n][m]`，而 n, m 最大是 2000，数组应该声明成 `[2001][2001]`。在 2000×2000 的满规模数据上原程序直接崩溃，ASan 报 `global-buffer-overflow`，lldb 里崩溃地址和 `&matrix[2000][...]` 完全吻合。官方测试点没有触发，只是数据没到边界。
2. **Bug B**: `int sum = 0;` 写在 `for (i = 1..q)` 外面，循环里也没有重置，所以第 i 个询问输出的是前 i 个询问答案之和。在询问入口打印 `sum`，能看到它带着上一轮的值继续累加。
3. **Bug C**: $2000 \times 2000 \times 10^5 = 4 \times 10^{11}$ 超过 `int` 上限 2.147×10^9 ，必须换成 `long long`，输出也改成 `%lld`。行内前缀和的最大值约 $2000 \times 10^5 = 2 \times 10^8$ ，还在 `int` 范围内，所以只需要把累加器和答案改成 64 位。
4. **Bug D**: `rowsum[i][j]` 表示第 i 行前 j 个元素之和，列区间 $[y, y+b-1]$ 应该是 `rowsum[i][y+b-1] - rowsum[i][y-1]`，原代码写成了 `rowsum[i][y+b] - rowsum[i][y]`，整个窗口右移了一列，还读到了未初始化的位置。

关于对拍能发现什么：`rand_input.cpp` 和 `check_input.cpp` 把数据限制在 n, m 从 10 到 100、 v 从 0 到 1000、 q 从 10 到 20，这样的数据撑不破数组边界，也累加不到 `int` 上限，所以对拍只能稳定复现 Bug B 和 Bug D。Bug A 和 Bug C 要靠静态分析、数据范围核算和自造的边界数据才能发现。

修复后的两份程序在官方 `1.in`、`2.in`、`3.in` 上都和标准答案逐行一致。

3B 调试时定位语句所需的编译选项

需要加 `-g`，例如 `g++ -g -o main main.cpp`，调试的时候一般再配上 `-O0`。`-g` 会让编译器把调试信息写进可执行文件，里面包括源文件名、函数名和变量名，以及每条机器指令的地址和源码行号的对应关系。调试器就是靠这张对应表，从当前停住的指令地址反查出源文件和行号，也才能按变量名读取栈里的值。不加 `-g`，可执行文件里只剩机器码，调试器只能看到地址和汇编。

提问方式：把 `rand_input.cpp`、`battle.cpp` 两个附件连同 3B 到 3E 四个问题一次性发给 LLM，要求逐条回答。对话记录和 0A 在同一个对话里：

[点击查看对话记录](#)

3C `srand(time(0))` 的作用

`srand` 用来给伪随机数发生器设置种子。`rand()` 产生的其实是一个固定的序列，如果不设种子，默认种子是 1，那么每次运行程序得到的随机数列完全一样，生成出来的测试数据也就每轮都相同。`time(0)` 是当前时间的秒数，用它当种子，不同时刻启动程序就会得到不同的序列，对拍才能一轮一轮产生新数据。这个调用只需要在程序开头执行一次。

[点击查看对话记录](#)

3D `battle.cpp` 中 `system` 调用的功能

`battle.cpp` 里的 `system` 调用一共 9 处。

1. `system("g++ rand_input.cpp -o rand_input")`：编译随机数据生成器；
2. `system("g++ check_input.cpp -o check_input")`：编译数据合法性检查程序；
3. `system("g++ solution_1.cpp -o solution_1")`：编译解法一；
4. `system("g++ solution_2.cpp -o solution_2")`：编译解法二；
5. `system("./rand_input > rand.in")`：运行生成器，用输出重定向把本轮数据写进 `rand.in`；
6. `system("./check_input < rand.in")`：把 `rand.in` 作为标准输入运行检查程序，检查数据是否满足约束，返回值非 0 时判定 `invalid input` 并跳出循环；
7. `system("./solution_1 < rand.in > 1.out")`：用同一份数据运行解法一，输出写进 `1.out`；
8. `system("./solution_2 < rand.in > 2.out")`：运行解法二，输出写进 `2.out`；
9. `system("diff 1.out 2.out")`：逐行比较两个输出，返回值非 0 说明两份结果不一致，打印 `different output!` 之后跳出循环，否则进入下一轮。

`system` 的返回值被用来控制流程。`check_input` 返回非 0 说明数据不合法，`diff` 返回非 0 说明两个程序结果不一致，这两种情况都会跳出 `while(1)`。`./rand_input > rand.in` 的返回值没有检查。整个程序就是一个自动对拍循环：生成数据、检查数据、分别运行两个程序、比较输出。

[点击查看对话记录](#)

3E 极端情况下的最大数值与数据类型

元素最多 $2000 \times 2000 = 4 \times 10^6$ 个，单个元素绝对值最大 10^5 ，所以子矩阵和的绝对值最大是

$$4 \times 10^6 \times 10^5 = 4 \times 10^{11}.$$

`int` 的上限约 2.15×10^9 ，只有这个值的约 $1/186$ ，一定会溢出；`long long` 的上限约 9.22×10^{18} ，放得下。所以前缀和和答案都要用 `long long`，输出用 `%lld`。矩阵元素本身可以用 `int` 读进来。数组的尺寸是另一回事：用 1-based 下标的时候 $n = m = 2000$ ，数组至少要开成 `[2001][2001]`。

[点击查看对话记录](#)

4 优化与比较

4A solution_3.cpp 的算法思路

做法是二维前缀和。 $\text{ans}[i][j]$ 表示第 1 行到第 i 行、第 1 列到第 j 列这个子矩阵的元素之和，用容斥可以在 $O(1)$ 时间递推出来：

$$\text{ans}[i][j] = v(i, j) + \text{ans}[i-1][j] + \text{ans}[i][j-1] - \text{ans}[i-1][j-1].$$

预处理整张表是 $O(nm)$ 。题目保证 $x+a-1 \leq n$ 、 $y+b-1 \leq m$ ，所以每次询问对应的就是第 x 到 $x+a-1$ 行、第 y 到 $y+b-1$ 列，一次容斥就能 $O(1)$ 回答：

$$\text{answer} = \text{ans}[x+a-1][y+b-1] - \text{ans}[x-1][y+b-1] - \text{ans}[x+a-1][y-1] + \text{ans}[x-1][y-1].$$

实现上有三点。数组开成 `long long ans[2001][2001]`，因为 1-based 下标最大到 2000，而且 4×10^{11} 的和必须用 64 位存；总复杂度是 $O(nm+q)$ ，和询问面积无关；输入最多有 4×10^6 个整数，用 `fread` 一次读完再自己解析，输出用 `printf`。

4B 运行时间还与哪些变量有关

当 n, m, q 固定时，三个程序的复杂度分别是 $O(nm+qab)$ 、 $O(nm+qa)$ 、 $O(nm+q)$ ，差别就在每次询问里还有哪些循环省不掉。

程序	总复杂度	n, m, q 固定后还和什么有关	原因
solution_1	$O(nm+qab)$	a, b	每次询问要访问整个 $a \times b$ 子矩阵，耗时正比于 $\sum a_i b_i$
solution_2	$O(nm+qa)$	a	每行的区间和 $O(1)$ 求出，宽度 b 被消掉，但仍要遍历 a 行，耗时正比于 $\sum a_i$
solution_3	$O(nm+q)$	无	每次询问固定 4 次数组访问，和 a, b, x, y 都无关

同样是 $a = 1000$ 的询问， $b = 10$ 和 $b = 1000$ 对 solution_2 的循环次数完全一样，对 solution_1 却差了 100 倍，而 solution_3 两种情况都只要 4 次数组访问。

除了算法本身，4C 的实测还说明下面这些也会影响用时。

1. 输入输出方式：输入有 4×10^6 个整数，大约 27 MB，读入时间可能比算法本身还长。同一份程序只把读入从 `cin` 换成 `fread`，用时从 3.2 s 降到 0.04 s。
2. 访存顺序和缓存：solution_1 逐行扫描子矩阵，solution_2 和 solution_3 顺序访问前缀和数组，缓存命中率更高。
3. 预处理的开销：solution_3 需要 $O(nm)$ 的预处理，还要占大约 32 MB 的数组，询问很少或者很小的时候这部分开销就不可忽略。
4. 编译选项和机器状态：-O2 和 -O0 差别很大；CPU 频率变化、其他进程占用、数据是不是还在页缓存里，都会让测量结果上下浮动，表现为极差。

LLM 对话记录链接：[点击查看对话记录](#)

4C 数据生成方式与运行用时

机器和编译环境：macOS (Darwin 25.6.0, Apple Silicon arm64)，g++ 即 Apple clang 21.0.0，编译选项 -O2。

数据生成方式：写了 `gen_data.cpp`，在 `rand_input.cpp` 的基础上扩大规模并改了 x, y, a, b 的生成方式，随机种子固定为 20261227，保证实验可复现。两类数据的规模相同： $n = m = 2000$ 、 $q = 10^5$ 、 $v(i, j)$ 在 -10^5 到 10^5 之间均匀随机，文件大约 27 MB。

1. 数据类 A： $x = 1$ 、 $a = n = 2000$ ，也就是每个询问都从第 1 行一直查到最后一行， y 随机、 b 不超过 20。此时 $\sum a_i b_i = 3.98 \times 10^9$ 、 $\sum a_i = 2.0 \times 10^8$ ，单次询问的面积在 2000 到 40000 之间。

2. 数据类 B: $a = b = 1$ 的单点询问, x, y 随机。此时 $\sum a_i b_i = \sum a_i = 10^5$, 和输入规模比起来可以忽略。

测量方法: 用 Python 脚本 `bench.py` 记每次运行的墙钟时间, 每类数据每个程序跑 5 次, 报告平均值和极差。数据文件放在本地磁盘, 同一组测量里所有程序共用同一份数据。

关于输入方式: 提交版 `solution_3` 用 `fread` 读入, `solution_1` 和 `solution_2` 用 `scanf`。为了把算法差异和读入差异分开, 另外编了一份算法完全一样、只把读入换成 `scanf` 的对照版本, 一起列在表里。

数据类	程序	平均用时/s	极差/s	备注
A	<code>solution_1</code>	2.575	0.183	朴素累加, 耗时正比于 $\sum a_i b_i = 3.98 \times 10^9$
A	<code>solution_2</code>	0.662	0.025	行前缀和, 耗时正比于 $\sum a_i = 2.0 \times 10^8$
A	<code>solution_3 对照版</code>	0.214	0.005	和提交版算法一样, 只把读入换成 <code>scanf</code>
A	<code>solution_3 提交版</code>	0.040	0.001	用 <code>fread</code> 读入
B	<code>solution_1</code>	0.215	0.008	询问面积只有 1, 算法开销可以忽略
B	<code>solution_2</code>	0.221	0.015	同上
B	<code>solution_3 对照版</code>	0.231	0.030	同上
B	<code>solution_3 提交版</code>	0.050	0.002	用 <code>fread</code> 读入

每组测量里几个程序输出的 md5 完全一致, 说明修复后的 `solution_1`、`solution_2` 和 `solution_3` 在满规模数据上结果是互相印证的。

本机是 Apple Silicon 加 Apple clang, 评测机是 Linux, 绝对时间不能直接比, 但两类数据下谁快谁慢的关系和平台无关。

4D 结论

数据类 A 上 `solution_3` 的优势很明显。都用 `scanf` 读入时, `solution_3` 只要 0.214 s, `solution_1` 是 2.575 s, `solution_2` 是 0.662 s, 分别是它的 12 倍和 3.1 倍。`solution_1` 的工作量正比于询问面积之和, `solution_2` 正比于询问高度之和, `solution_3` 每次询问固定 4 次数组访问。

数据类 B 上 `solution_3` 没有优势。三个程序都在 0.2 s 左右, 分别是 0.215 s、0.221 s、0.231 s, 差别不到 8%。这时时间主要花在读入 4×10^6 个整数和预处理上, 算法本身的差别看不出来。

所以 `solution_3` 的优势要看询问面积之和是否大到超过读入和预处理的固定开销。本题满规模数据下, 只要询问的子矩阵平均尺寸比较大, 优势就明显。

读入方式的影响比预想的大。用 `cin` 的那一版 (已经关了同步) 在本机读 27 MB 要 3.2 s, 提交到 OJ 之后超时了; 换成 `scanf` 是 0.21 s, 改成 `fread` 之后是 0.04 s, 两类数据上它都是三个程序里最快的。

5 AI 工具使用

5A `solution_3` 算法的 HTML 可视化网页

LLM 对话记录链接: [点击查看对话记录](#)

我让 LLM 生成一个单独的 HTML 网页, 要求 HTML、CSS、JS 全部内嵌, 不依赖任何外部资源, 用来演示二维前缀和的构造和查询。网页可以自己输入或者随机生成一个小矩阵, 同时显示原矩阵和前缀和矩阵; 构造过程可以单步或者自动播放, 高亮当前算的格子和它用到的三个格子, 并把四项数值显示出来; 输入 x, y, a, b 之后会一步一步演示容斥的加减过程; 播放速度可以调。



图 1 可视化网页截图。用题目样例的 4×4 矩阵，构造完成后演示查询 $x = 2, y = 2, a = 3, b = 3$ ，结果是 48，和题面样例一致。

完整代码见附录 B，也可以直接打开随报告提交的 `prefix_sum_visualizer.html`。

5B 让 LLM 给本报告打分与修改建议

我用的提示词：

你是一名经验丰富的数据结构课程助教。下面是我 Lab1 (Embrace DSA) 的实验报告全文。

请：1) 按内容完整性、思考深度、实验严谨性打分（百分制）并说明扣分点；
2) 给出具体的修改建议（不必替我改写）；3) 指出我报告中论证不充分或缺失的部分。

LLM 对话记录链接：[点击查看对话记录](#)

它给的总分是 91 / 100，分项是内容完整性 95、思考深度 95、实验严谨性 83、表达与结构 90。它认为报告把 0A 到 5B 的任务都覆盖了，2C 里对 LLM 输出可靠性的分析、3A 里对“哪些 bug 对拍发现不了”的分类、以及 4B 的复杂度分析都超出了题目的基本要求。扣分主要在实验严谨性，理由是部分结论的证据链偏弱，性能实验的数据覆盖范围也不够。它认为补齐它列出的必改项之后可以到 95 分左右。

按题目要求，这些建议没有实施，只附在报告最后，见附录 D。

附录

附录 A：LLM 对话记录链接汇总

任务	链接
0A	link
2C	link
3B	link
3C	link
3D	link
3E	link
4B	link
5A	link
5B	link

附录 B：可视化网页代码

下面是 LLM 生成的 prefix_sum_visualizer.html 完整代码，保存后用浏览器打开即可运行。

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<title>二维前缀和可视化</title>
<style>
body{
    font-family:"Microsoft YaHei",sans-serif;
    background:#f5f6fa;
    margin:20px;
}
h1{
    text-align:center;
}
.panel{
    background:white;
    padding:15px;
    border-radius:12px;
    margin-bottom:20px;
    box-shadow:0 2px 8px #ccc;
}
input,button{
    padding:6px 10px;
    margin:4px;
}
button{
    cursor:pointer;
}
.tables{
    display:flex;
    gap:30px;
    flex-wrap:wrap;
}
table{
    border-collapse:collapse;
}
td{
```

```

        width:55px;
        height:45px;
        text-align:center;
        border:1px solid #999;
        position:relative;
    }
    .head{
        background:#eee;
        font-weight:bold;
    }
    .value{
        font-size:14px;
    }
    .prefix{
        font-size:13px;
        color:#555;
    }

    .current{
        background:#ffcc66!important;
    }
    .dep{
        background:#9ddcff!important;
    }
    .query{
        background:#ff9999!important;
    }

```

```

    .info{
        font-size:18px;
        min-height:50px;
        padding:10px;
        background:#eef;
        border-radius:8px;
    }

```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>二维前缀和可视化</h1>
```

```
<div class="panel">
```

```
n:
```

```
<input id="n" value="6" type="number">
```

```
m:
```

```
<input id="m" value="8" type="number">
```

```
<button onclick="generate()">随机生成</button>
```

```
<button onclick="reset()">重置</button>
```

```
<br>
```

速度:

```
<input id="speed" type="range" min="200" max="2000" value="800">
```

```
<button onclick="step()">单步</button>
```

```
<button onclick="play()">播放</button>
```

```
</div>
```

```
<div class="panel">
```

查询:

x=

```
<input id="qx" value="2" type="number">
```

y=

```
<input id="qy" value="2" type="number">
```

a=

```
<input id="qa" value="3" type="number">
```

b=

```
<input id="qb" value="4" type="number">
```

```
<button onclick="queryDemo()">开始查询演示</button>
```

```
</div>
```

```
<div class="panel">
```

```
<div id="info" class="info">
```

等待操作...

```
</div>
```

```
</div>
```

```
<div class="tables">
```

```
<div>
```

```
<h3>原矩阵 v(i,j)</h3>
```

```
<div id="matrix"></div>
```

```
</div>
```

```
<div>
```

```
<h3>二维前缀和 ans[i][j]</h3>
```

```
<div id="prefix"></div>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
let n=6,m=8;
```

```
let mat=[];
```

```
let pre=[];
```

```
let ci=1,cj=1;
```

```
let timer=null;
```

```
let mode="build";
```

```

function generate(){

    n+=document.getElementById("n").value;
    m+=document.getElementById("m").value;

    mat=[];

    for(let i=0;i<n;i++){
        mat[i]=[];
        for(let j=0;j<m;j++){
            mat[i][j]=Math.floor(Math.random()*19)-9;
        }
    }

    reset();
}

```

```

function reset(){

    pre=Array.from(
        {length:n+1},
        ()=>Array(m+1).fill(0)
    );

    ci=1;
    cj=1;
    mode="build";

    draw();

    info("准备计算 ans[1][1]");
}

```

```

function info(t){
    document.getElementById("info").innerHTML=t;
}

```

```

function draw(){

    drawMatrix();
    drawPrefix();

}

```

```

function drawMatrix(){

let html="<table>";

for(let i=0;i<n;i++){

```

```

html+="|";

for(let j=0;j<m;j++){

let cls="";

if(mode=="query" &&
    queryCells.some(x=>x[0]==i+1&&x[1]==j+1))
cls="query";

html+=`
<td class="${cls}">
<div class="value">${mat[i][j]}</div>
</td>`;

}

html+="</tr>";

}

html+="</table>";

document.getElementById("matrix").innerHTML=html;

}

|  |

```

```

let highlights=[];

```

```

function drawPrefix(){

let html="<table>";

for(let i=0;i<=n;i++){

html+="|";

for(let j=0;j<=m;j++){

let cls="";

if(highlights.some(x=>x[0]==i&&x[1]==j))
cls=i==ci&&j==cj?"current":"dep";

html+=`
<td class="${cls}">
<div class="prefix">
${pre[i][j]}
</div>
</td>`;

|  |

```

```

}

html+="|";

}

html+="




|  |

```

```

function step(){

if(mode!="build") return;

if(ci>n){
    info("二维前缀和构造完成");
    return;
}

let v=mat[ci-1][cj-1];

let up=pre[ci-1][cj];
let left=pre[ci][cj-1];
let cross=pre[ci-1][cj-1];

pre[ci][cj]=
v+up+left-cross;

highlights=[
[ci,cj],
[ci-1,cj],
[ci,cj-1],
[ci-1,cj-1]
];

draw();

info(`
计算:
<br>
ans[${ci}][${cj}]
=
${v}
+
ans[${ci-1}][${cj}](${up})
+
ans[${ci}][${cj-1}](${left})
-
ans[${ci-1}][${cj-1}](${cross})

```

```
=  
${pre[ci][cj]}  
`);
```

```
cj++;
```

```
if(cj>m){  
    cj=1;  
    ci++;  
}  
  
}
```

```
function play(){
```

```
if(timer){  
clearInterval(timer);  
timer=null;  
return;  
}
```

```
timer=setInterval(()=>{
```

```
step();
```

```
if(ci>n){  
clearInterval(timer);  
timer=null;  
}
```

```
},+document.getElementById("speed").value);
```

```
}
```

```
let queryCells=[];
```

```
function queryDemo(){
```

```
mode="query";
```

```
let x+=qx.value;  
let y+=qy.value;  
let a+=qa.value;  
let b+=qb.value;
```

```
let A=[x+a-1,y+b-1];  
let B=[x-1,y+b-1];  
let C=[x+a-1,y-1];  
let D=[x-1,y-1];
```

```
queryCells=[A,B,C,D];
```

```
highlights=queryCells;
```

```
draw();
```

```
let value=  
pre[A[0]][A[1]]  
-pre[B[0]][B[1]]  
-pre[C[0]][C[1]]  
+pre[D[0]][D[1]];
```

```
let steps=[
```

```
\
```

```
取大矩形:
```

```
ans[ $\{A\}$ ] =  $\{pre[A[0]][A[1]]\}$ 
```

```
\
```

```
,
```

```
\
```

```
减去上方区域:
```

```
- ans[ $\{B\}$ ] = - $\{pre[B[0]][B[1]]\}$ 
```

```
\
```

```
,
```

```
\
```

```
减去左侧区域:
```

```
- ans[ $\{C\}$ ] = - $\{pre[C[0]][C[1]]\}$ 
```

```
\
```

```
,
```

```
\
```

```
加回重复减去区域:
```

```
+ ans[ $\{D\}$ ] = + $\{pre[D[0]][D[1]]\}$ 
```

```
<br>
```

```
答案= $\{value\}$ 
```

```
\
```

```
];
```

```
let k=0;
```

```
let t=setInterval(()=>{
```

```
info(steps[k]);
```

```
k++;
```

```
if(k==steps.length)
```

```
clearInterval(t);
```

```
},800);
```

```
}
```

```
generate();
```

</script>

</body>

</html>

附录 C: 提交检查清单

- ☐ 0A、2C、3A 3E、4A 4D、5A 5B 全部覆盖;
- ☐ 所有 LLM 对话链接在未登录状态下可正常打开;
- ☐ solution_3.cpp 即 t1.cpp 通过官方 3 个测试点, 提交黑盒, 标记 Final Version, 签署 Honor Code;
- ☐ 报告内不粘贴大段聊天记录, 只保留提示词和结论;
- ☐ 不含编译产物, 文件组织清晰;
- ☐ 含截图, 用 PDF 提交。

附录 D: LLM 给出的修改建议 (未实施)

LLM 助教按收益排序给出的建议如下。

必改

1. 补一组中间规模的性能实验数据。现在只测了大面积询问和单点询问两个极端, 建议再加一组 a, b 在 100 到 500 之间的数据, 看 solution_2 和 solution_3 的分界点在哪里。
2. 增加调试证据摘要。3A 里 ASan、lldb、UBSan、对拍的关键输出没有直接给出, 建议加一张表, 列出每个 bug 用了什么工具、关键输出是什么, 证明确实做过这些验证。
3. 增加空间复杂度分析。三个程序的空间都是 $O(nm)$, 但维护的信息不一样, 建议补一张时间和空间的对照表, 说明空间换时间的关系。
4. 补充对拍结论的限制。两个程序输出一致不等于正确, 如果它们有同一个 bug, 对拍也会通过。建议把这一点写出来, 并补充和标准程序比较、手算小数据等验证方式。
5. 给二维前缀和的容斥补一张示意图, 说明大矩形减去上方、减去左方、再加回重复减掉的部分。

可选

1. 输入优化的篇幅偏多, 容易把主题从算法复杂度拉向工程优化, 建议加一句说明输入方式不影响二维前缀和在询问复杂度上的理论优势。
2. 性能实验每个程序只跑 5 次, 对 0.04 s 这个量级来说偏少, 建议跑 10 次并去掉第一次冷启动, 或者说明是用极差来判断稳定性的。
3. 表达上有些地方更像技术博客而不是课程报告, 可以再收一收。